TCS Corp - Developer Guide

Date: October 11, 2026

Adobe

Table of Contents

TCS Corp - Developer Guide 3
Quick Reference 3
Architecture Overview 3
Custom Page Decoration (scripts/scripts.js) 6
Site Structure and Site Data 7
Conventions 12
Shared Code: Decorators and Utils 13
Local Development Setup 15
Design System 16
Blocks Reference 18
Integrations and Delayed Phase 21
Universal Editor Models 22
Common Development Tasks 22
Environments 24
Git Workflow 24
Troubleshooting 24
Known Gaps and Gotchas 25
Resources 26
Support Contacts 27

TCS Corp - Developer Guide

Technical guide for developers maintaining and extending the TCS Corp website, built with AEM Edge Delivery Services and authored in Universal Editor (xwalk).

Quick Reference

Resource	URL
Code Repository	https://github.com/sradha-tcs/tcs-corp
Preview	https://main--tcs-corp--sradha-tcs.aem.page/
Live	https://main--tcs-corp--sradha-tcs.aem.live/
Local Dev	http://localhost:3000
Content Source	AEM as a Cloud Service author `author-p220207-e2271469.adobeaemcloud.com` (via `fstab.yaml`)
Config Service Org	`sradha-tcs` (placeholder until the final organization name is decided)

> **Config Service status:** The site list request (`https://admin.hlx.page/config/sradha-tcs/sites.json`) returned **HTTP 401 (not authenticated)** when this guide was generated, so the list of sites could not be checked. Preview and live both respond for `tcs-corp`, so this guide treats the project as a **standard single-site setup**. Once the final organization name is set, confirm with an authenticated Admin API call. If more sites are added (repoless), start the local server with `aem up --pages-url=https://main--{site}--{org}.aem.page`.

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+ modules), no transpiling and no build step
- CSS3 with custom properties (TCS.com Design System tokens), no CSS framework
- Content authored in **AEM Universal Editor** (xwalk) and delivered through the AEM author mountpoint
- 13 locale sites: AEM `/content/tcs-corp/{country}/{lang}` (e.g. `de/de-de`) published at language-only URLs (`/en`, `/de-de`, ...). Each locale site has its own references (nav, footer, fragments), error pages, placeholders, metadata, taxonomy, data sheets and block library
- Site data from standard Edge Delivery Services sources: page metadata, the bulk metadata sheet, placeholders, taxonomy, redirects and data spreadsheets. Page properties are limited to Title, Description, Keywords, Page Type, Navigation and Footer
- Vendored libraries: `lottie-web` light player (`scripts/lib/lottie/`) and DOMPurify (`scripts/dompurify.min.js`, used only by `editor-support.js`)
- Tooling: ESLint (airbnb-base + `eslint-plugin-xwalk`), Stylelint (standard), Husky pre-commit hook

Project Structure

```
├── blocks/              # 24 blocks (21 customized for TCS incl. header and footer, 3
boilerplate)
├── decorators/          # 17 single-purpose UI builders (see decorators/README.md)
├── utils/               # 17 helpers that build no UI (see utils/README.md)
├── templates/           # Site Global Template code by Page Type: home, article, blog,
press-release
├── models/              # Universal Editor models: page, section, button, ...
├── scripts/
│   ├── aem.js           # Core library (DO NOT MODIFY)
│   ├── scripts.js       # Page decoration: head (SEO), templates, error pages, sections,
modals
│   ├── editor-support.js   # Universal Editor live re-decoration
│   ├── editor-support-rte.js # Boilerplate rich-text grouping for the editor
│   ├── delayed.js       # Delayed phase (boilerplate, no integrations yet)
│   ├── dompurify.min.js # Sanitizer for editor rich-text updates
│   └── lib/lottie/      # Vendored lottie-web light player
├── styles/
│   ├── styles.css       # Design tokens, buttons, tags, avatars, sections
│   ├── fonts.css        # Inter variable font
│   └── lazy-styles.css  # Non-critical styles (empty)
├── icons/               # 21 SVG icons (arrows, chevrons, play, pause, mic, share, social
platforms, ...)
├── fonts/               # inter-variable.woff2
├── 404.html             # Error page: loads the locale's authored error-pages/404
├── paths.json           # Generated from LOCALES (npm run build:paths): locale, sheet and
taxonomy mappings, includes, excludes
├── tools/paths/         # generate-paths.mjs (writes paths.json)
├── tools/sidekick/      # Sidekick config.json (boilerplate)
├── helix-query.yaml     # Query index: content pages and metadata sheet columns
├── component-*.json     # Generated from models/ and blocks/*/_*.json (npm run
build:json)
└── head.html            # CSP (nonce "aem", strict-dynamic) and core scripts
```

The **Page Type** (page property, else the metadata sheet's `page-type` column) becomes a body class, selects the template code in `templates/` and drives the Page Header layout (see *Site Global Templates*).

Three-Phase Loading (E-L-D)

Phase	Purpose	What Loads	Performance Impact
Eager	Render above-the-fold content (LCP)	`styles/styles.css`, `scripts.js`, `decorateHead` (SEO tags, `lang`), `decorateTemplateAndTheme`, Page Type class, the template's JS/CSS or the authored error page, `decorateMain`, first section and its blocks	Blocks LCP, keep it small
Lazy	Remaining content after first paint	Modal link handler, header, other sections, footer, `lazy-styles.css`, `fonts.css`	Safe for non-critical UI
Delayed	Non-essential third-party scripts	`scripts/delayed.js` (3 s after load)	Never blocks rendering

Rules for developers:

- **Never add third-party scripts to `scripts.js`**, as they block LCP. Use `delayed.js`.

- `fonts.css` loads eagerly only on desktop (>= 900px) or when `sessionStorage['fonts-loaded']` is already set. Otherwise it loads in the lazy phase.
- Blocks in the first section load eagerly. All other blocks load lazily, based on their position in the DOM.
- Site data sheets (the locale's `placeholders.json`, `taxonomy.json` and `data/*.json`) are fetched **once per page** and only by code that needs them. Templates load sheet data without blocking (`appendLoadedSection`). A block in the first section that awaits them delays LCP by one request, so keep these lookups small.
- Heavy media is deferred inside blocks: the Lottie player loads only when the hero is near the viewport and the browser is idle. Embeds and videos use `onVisible()`.

Key Files and How They Connect

File	Role	Connects To
`scripts/aem.js`	Core library. **DO NOT MODIFY.**	Imported by scripts, blocks, decorators, utils
`scripts/scripts.js`	Entry point: `loadEager` -> `loadLazy` -> `loadDelayed`, head decoration, templates, error pages, `decorateMain`, section options, Gen Carousel auto-blocking, modal links	`aem.js`, `decorators/seo-decorator.js`, `decorators/responsive-image-decorator.js`, `decorators/content-buttons-decorator.js`, `utils/site.js`, `templates/`
`templates/{name}/{name}.js` / `.css`	Structure and styles a Site Global Template adds before decoration	decorators, utils
`scripts/editor-support.js`	Re-decorates content after Universal Editor changes (uses `decorateSectionsWithOptions`, `decorateButtons`)	`scripts.js`, DOMPurify
`decorators/*.js`	Create or decorate UI elements the same way in every block	May import `utils/`
`utils/*.js`	Helpers that build no UI, incl. the locale site and site data (site, metadata, placeholders, taxonomy, data sheets)	**Never import decorators**
`styles/styles.css`	Tokens and global components (buttons, tags, avatars, sections)	All blocks
`blocks/{name}/{name}.js` / `.css`	Block decoration and styles, loaded automatically	decorators, utils, `scripts.js`
`blocks/{name}/_{name}.json`	Universal Editor definition, model, filter	Merged into `component-*.json`

Execution flow: page load -> `scripts.js` -> `loadEager()` (`decorateHead`, `decorateTemplateAndTheme`, `decoratePageType`, `loadErrorPage` on 404 or `loadTemplate`, `decorateMain`, first section) -> `loadLazy()` (modal links, header, sections, hash scroll, footer, lazy CSS, fonts) -> `loadDelayed()` (`delayed.js` after 3 s).

Functions Imported from aem.js

`scripts.js` imports `loadHeader`, `loadFooter`, `decorateIcons`, `decorateSections`, `decorateBlock`, `decorateBlocks`, `decorateTemplateAndTheme`, `waitForFirstImage`, `loadSection`, `loadSections`, `loadCSS`,

`toClassName`, `getMetadata`. `utils/metadata.js` uses `getMetadata`, `decorators/responsive-image-decorator.js` uses `createOptimizedPicture`. Blocks mainly use `toClassName`, `getMetadata` and `loadScript`.

Custom Page Decoration (scripts/scripts.js)

decorateMain

```
export function decorateMain(main) {
  decorateIcons(main);
  buildAutoBlocks(main);             // empty placeholder (TODO)
  decorateSectionsWithOptions(main); // section options + Gen Carousel sections
  decorateBlocks(main);
  decorateButtons(main);             // decorators/content-buttons-decorator.js
}
```

Head, Page Type, Templates and Error Pages (loadEager)

```
async function loadEager(doc) {
  decorateHead();             // decorators/seo-decorator.js: SEO / social columns, <html
lang>
  decorateTemplateAndTheme(); // core: template / theme body classes
  decoratePageType();         // page-type body class
  const main = doc.querySelector('main');
  if (main) {
    if (window.isErrorPage) await loadErrorPage(main); // 404.html
    else await loadTemplate(main);                     // templates/{name}/
    decorateMain(main);
    ...
```

- **`decorateHead()`** maps the custom metadata-sheet columns to the standard head tags: `meta-title` -> `<title>`, `meta-description` / `meta-keywords` -> description / keywords, `og-*` -> `og:*` (falling back to `meta-*`), `twitter-*` -> `twitter:*` (falling back to `og-*`), `canonical-url` -> `<link rel="canonical">` and `og:url`, `language` (else the locale's language tag, e.g. `de-DE`) -> `<html lang>`. These are applied in the browser; social crawlers that don't run JavaScript only see server-side tags, so the standard column names (`og:title`, `canonical`, ...) are needed for social previews.
- **`loadTemplate(main)`** reads the Page Type (`page-type`, else `template`). If it is in `TEMPLATES` (`home`, `article`, `blog`, `press-release`), it imports `templates/{name}/{name}.js` and loads its CSS in parallel, then calls the template's default export with `main` **before** decoration, so templates can add sections and blocks.
- **`loadErrorPage(main)`** (404.html sets `window.isErrorPage`) fetches the locale's `error-pages/{errorCode}.plain.html` and replaces the built-in content, rebasing `./media_` images. When the page isn't published, the built-in content stays and its "Go home" button points to the locale home.

Section Options (`decorateSectionsWithOptions`)

The section model (`models/_section.json`) adds fields that are published as section-metadata rows. `decorateSections()` from `aem.js` keeps only the URL of a metadata image. Because of this, `scripts.js` reads and removes these rows **before** it calls `decorateSections()`, then applies them to the section:

Field (model name)	Effect on the section
Width Type (`sectionWidth`)	`fixed-width` class, so the content uses `--content-max-width` (1440px). Full width (default) uses `--page-max-width` (1920px)

Field (model name)	Effect on the section
Rounded edge (`roundedEdge`)	`rounded` class (`--radius-l`, no margin)
Background Image (`backgroundImage`) + Image alt text (`backgroundImageAlt`)	Prepends `.section-background` with an image from `buildImage()` (site media optimized at 750 / 2000 widths) and adds `has-background`. An empty alt marks the image as decorative
Anchor name (`anchorName`)	Sets the section `id` (only if it isn't already used). Anchor List links jump to it
Style (`style`)	Standard section class. The model offers **Highlight** only

Editor support calls the same function, so section options also apply after live edits in Universal Editor.

Gen Carousel Sections (auto-blocking)

`blocks/gen-carousel/_gen-carousel.json` defines **Gen Carousel** as a *section* (resource type `section/v1/section`, style `gen-carousel-section`). Its filter allows `hero` and `teaser`, and the main filter allows `section` and `gen-carousel`. `buildCarouselSections()`:

1. Finds `.section.gen-carousel-section` sections.
2. Creates a synthetic `div.gen-carousel` block. Navigation (`manual`, `auto`, `manual-auto`) becomes a class and the interval becomes `data-interval`.
3. Calls `decorateBlock()` on the carousel while it is still empty, so the slides are not treated as cells.
4. Decorates each child block itself, because `decorateBlocks()` only finds direct section children, then moves every wrapper into the carousel as a slide.

Modal Links

`autolinkModals()` handles every click on a link whose path contains `/modals/`. It imports `blocks/modal/modal.js` and opens the fragment in a `<dialog>`. Links that are already handled (`defaultPrevented`) are skipped.

Exported Helpers

`moveAttributes`, `moveInstrumentation` (moves `data-aue-*` / `data-richtext-*` attributes when blocks replace elements; use it in every block that rebuilds markup), `decorateSectionsWithOptions`, `decorateMain`, `decorateButtons` (re-export).

Site Structure and Site Data

Content Tree and Locales

```
/content/tcs-corp/
├── us/en/                  # a locale site (country/lang), published at /en
│   ├── index               # Home and the content pages
│   ├── references/         # nav (header), footer and other fragments / reference pages
│   ├── error-pages/        # 404, 403, 500
│   ├── placeholders        # sheet: Key, Text, Description, Category
│   ├── metadata            # sheet: bulk metadata (published as /en/metadata.json)
│   ├── taxonomy            # taxonomy page (tag, title; a sheet per language when
translated)
│   ├── redirects           # sheet: source, destination (published as /en/redirects.json,
not applied by the platform)
│   ├── data/               # sheets: authors, locations, partners, social-links, media-
contacts
```

```
│   └── block-library/      # pages with every block variation, for testing on author; not
published
├── de/de-de/ ... kr/ko-kr/ # the other 12 locale sites, same layout
└── site-global-templates/  # template pages authors copy (home, press release, blog,
article); not published
```

The 13 locale sites (`LOCALES` in `utils/site.js`; `paths.json` is generated from it):

AEM folder (`/content/tcs-corp/...`)	Published root	`<html lang>`	Taxonomy sheet
us/en	/en (default)	en-US	en
de/de-de	/de-de	de-DE	de
fr/fr-fr	/fr-fr	fr-FR	fr
ca/fr-ca	/fr-ca	fr-CA	fr
es/es-es	/es-es	es-ES	es
pt/pt-pt	/pt-pt	pt-PT	pt
nl/nl-nl	/nl-nl	nl-NL	nl
se/sv-se	/sv-se	sv-SE	sv
dk/da-dk	/da-dk	da-DK	da
no/no-no	/no-no	no-NO	no
cn/zh-cn	/zh-cn	zh-CN	zh
jp/ja-jp	/ja-jp	ja-JP	ja
kr/ko-kr	/ko-kr	ko-KR	ko

`utils/site.js` resolves the locale of a page **from its AEM path when editing** (`/content/tcs-corp/de/de-de/...`) and **from its published path otherwise** (`/de-de/...`). Pages outside a locale (`site-global-templates`, `drafts`) use `DEFAULT_LOCALE` (en).

Export	Returns
`LOCALES`, `DEFAULT_LOCALE`	The locale list (`{ country, code, lang }`, e.g. `{ 'de', 'de-de', 'de-DE' }`) and its first entry (en)
`getLocale(pathname?)`	`{ country, code, lang, language, root, aemRoot, contentRoot }`, e.g. `{ 'de', 'de-de', 'de-DE', 'de', '/de-de', '/de/de-de', '' }`. `language` is the ISO 639-1 code; `contentRoot` is `/content/tcs-corp` when editing, else `''`
`toPublishedPath(aemPath)`	`/content/tcs-corp/de/de-de/about` -> `/de-de/about` (following `paths.json`); other paths unchanged
`toAemPath(publishedPath, contentRoot)`	`/de-de/about` -> `/content/tcs-corp/de/de-de/about`; AEM paths unchanged
`localePagePath(path)`	A page of the locale: `references/nav` -> `/de-de/references/nav` on the site, `/content/tcs-corp/de/de-de/references/nav` when editing
`resolvePagePath(value, fallback)`	A configured page path (AEM path, published path or locale-relative path) converted to the path to fetch here (AEM path when editing, published path on the site), else the locale's fallback page

Export	Returns
`localeSheetPath(name)`	A sheet URL of the locale, e.g. `placeholders` -> `/de-de/placeholders.json`
`sheetRows(json, sheet?)`	Rows of a sheet JSON; for a multi-sheet JSON the named sheet, else the default

`utils/urls.js` `toSitePath()` uses `toPublishedPath()`, so authored links to AEM pages (`/content/tcs-corp/de/de-de/about.html`) become published paths (`/de-de/about`).

Where the Data Comes From

Data	Source (published as)	Code
Header / Footer	Navigation / Footer page properties, else the `nav` / `footer` columns of the metadata sheet, else the locale's `references/nav` / `references/footer`	`blocks/header`, `blocks/footer` with `resolvePagePath()`
Templates	Page Type page property, else the metadata sheet's `page-type`	`loadTemplate()`, `templates/`
Error pages	`error-pages/404` (403 / 500 are regular pages a CDN can serve)	`loadErrorPage()`
Page metadata	Page properties and the locale's bulk metadata sheet (`/en/metadata.json`, `/de-de/metadata.json`, ...; applied once all 13 are listed in the site configuration's `metadata.source`)	`utils/metadata.js`, `decorators/seo-decorator.js`
UI text	`/{code}/placeholders.json` (e.g. `/de-de/placeholders.json`)	`utils/placeholders.js`
Tag labels	`/{code}/taxonomy.json` (mapped per locale; the sheet named by the ISO 639-1 language, e.g. de, when translated)	`utils/taxonomy.js`
Data sheets	`/{code}/data/*.json` (e.g. `/de-de/data/authors.json`)	`utils/data.js`
Redirects	The locale's `redirects` sheet (`/{code}/redirects.json`, e.g. `/de-de/redirects.json`)	None. The platform only applies a root `/redirects.json`, so these aren't applied automatically; redirects (incl. `/` -> `/en/`) need another mechanism, e.g. CDN rules (see *Known Gaps*)

Page Metadata (utils/metadata.js)

Bulk metadata columns (each published as `<meta name="{column}">`): `url`, `page-type`, `meta-title`, `meta-description`, `meta-keywords`, `author-id`, `publish-date`, `modified-date`, `reading-time`, `og-title`, `og-description`, `og-image`, `og-type`, `twitter-card`, `twitter-title`, `twitter-description`, `twitter-image`, `canonical-url`, `robots`, `region`, `country`, `language`, `persona`, `thumbnail-image`, `featured-image`, `campaign-id`, `content-type`, `featured`, `priority`, plus `nav` and `footer`. Page-level values (page properties) win over the sheet.

- `getPageMetadata()` returns `{ title, description, image, pageType, contentType, category, tags, authorId, author, publishDate, modifiedDate, readingTime, thumbnailImage, featuredImage, region, country, language, persona, campaignId, featured, priority }`. `title` and `description`

are the **authored** values, read before the SEO decorator replaces them with `meta-title` / `meta-description`. `pageType` falls back to `template`, `publishDate` to `publication-date`, `country` to the locale; `featured` is a boolean (yes / true / 1), `priority` a number or `null`.

- `getPageTags()` reads every `<meta property="article:tag">`, falling back to a comma-separated `tags` meta.
- `getPageLanguage()` returns the `language` metadata when it is a language code, else the locale's language tag (e.g. `de-DE`); it sets `<html lang>`.

Placeholders (utils/placeholders.js)

```
import { getPlaceholders } from '../../utils/placeholders.js';

const t = await getPlaceholders();
button.setAttribute('aria-label', t('nextSlide', 'Next slide'));
label.textContent = t('slideOf', '{n} of {total}', { n: 1, total: 4 });
```

- `loadPlaceholders()` fetches the locale's `placeholders.json` **once per page** (columns **Key** and **Text**; Description and Category are notes for authors). A missing sheet gives an empty map, so the fallbacks are used.
- Keys are **normalized** (lowercase, no separators), so `Pause Slides`, `pause-slides` and `pauseSlides` all match `pauseSlides`. Use camelCase in code.
- `{name}` variables are filled by `fillPlaceholder(text, values)`.
- **Always pass an English fallback.**

Keys used in code:

Area	Keys
Carousel	`carousel`, `previousSlide`, `nextSlide`, `pauseSlides`, `playSlides`, `showSlide` (`{n}`, `{total}`), `slideOf` (`{n}`, `{total}`)
Page Header	`published`, `minRead` (`{n}`), `listenToThisPage`, `downloadThisPage`, `shareThisPage`, `linkCopied`
Header / Footer	`openNavigation`, `closeNavigation`, `followUsOn` (`{platform}`)
Templates	`aboutTheAuthor`, `viewLinkedinProfile`, `mediaContacts`
Anchor List	`onThisPage`
Conversation Canvas	`searchPlaceholder`, `searchByVoice`, `send`, `suggestions`, `noPagesFound`, `menu`, `siteNavigation`, `back`
Table List	`card`, `exploreItem` (`{title}`)
Forms	`submit`, `selectAnOption`, `formSuccess`, `shownWhenValue` (`{value}`, editor only)
Embed / Video / Modal	`play`, `playVideo`, `close`

Taxonomy (utils/taxonomy.js)

- `loadTaxonomy()` fetches the locale's `taxonomy.json` once per page into a map of tag ID to title. For a multi-sheet taxonomy (translated titles), the sheet named by the locale's ISO 639-1 language (`getLocale().language`, e.g. `de`, `fr`, `ja`) is used, else the default sheet.
- `resolveTags(values)` returns `{ id, label }[]`, with **ID-derived labels** (`parseTag()`) for tag IDs missing from the taxonomy.

- Used by the Teaser, the Page Header and the Conversation Canvas.

Data Sheets (utils/data.js)

Sheet	Columns	Reader	Used by
`data/authors`	author-id, name, designation, bio, image, linkedin, region	`getAuthors()`, `getAuthor(id)` (case-insensitive ID)	Page Header (author-id), article / blog templates
`data/locations`	location-id, location-name, country, region, latitude, longitude	`getLocations()` (numbers for coordinates)	Not displayed yet
`data/partners`	partner-id, partner-name, logo, website, tier, description	`getPartners()`	Not displayed yet
`data/social-links`	platform, url, icon, sort-order	`getSocialLinks()` (in sort order)	Footer
`data/media-contacts`	id, name, designation, email, phone, region	`getMediaContacts()`, `inRegion(rows, region)`	Press release template

Each sheet is loaded once per page (`loadDataSheet(name)`); rows become camelCase objects and missing sheets give empty lists.

Site Global Templates (templates/)

Template	Page Type	Adds
`home/`	`home`	Layout styles only (the first section meets the header)
`article/`	`article`	A Page Header from the page metadata unless one is authored (`ensurePageHeader`); "About the author" from the Authors sheet (`appendLoadedSection` + `buildAuthorBio`)
`blog/`	`blog`	The article structure; `blog.css` imports the article styles
`press-release/`	`press-release`	A Page Header unless authored; "Media contacts" from the Media Contacts sheet, filtered by the page's `region` (else all)

The template pages in `site-global-templates/` are copied by authors to create pages; they set the Page Type. See *Add a Template*.

Path Mapping (paths.json)

`paths.json` is **generated** by `npm run build:paths` (`tools/paths/generate-paths.mjs`), which loads `LOCALES` from `utils/site.js`. Don't edit it by hand. 130 mappings, 13 includes, 28 excludes:

```
mappings (130):
  # 13 locale folders first, e.g. /content/tcs-corp/de/de-de/:/de-de/
  /content/tcs-corp/{country}/{lang}/:/{code}/
```

```
  # then per locale (13 x 9), the more specific mappings (the last match wins):
  # 8 spreadsheets, e.g. .../de/de-de/data/authors:/de-de/data/authors
  /content/tcs-corp/{country}/{lang}/{sheet}:/{code}/{sheet}
  #   {sheet} = placeholders, metadata, redirects, data/authors, data/locations,
  #             data/partners, data/social-links, data/media-contacts
  # 1 taxonomy page, e.g. .../de/de-de/taxonomy:/de-de/taxonomy.json
  /content/tcs-corp/{country}/{lang}/taxonomy:/{code}/taxonomy.json
includes (13):
  /content/tcs-corp/{country}/{lang}/          # 13 locale roots
excludes (28):
  /content/tcs-corp/{country}/{lang}/block-library(/**)   # 13 x 2
  /content/tcs-corp/site-global-templates(/**)            # 2
```

- **13 locale mappings:** each AEM locale folder is published at its language-only root (`us/en` -> `/en`, `de/de-de` -> `/de-de`, ..., `kr/ko-kr` -> `/ko-kr`).
- **104 spreadsheet mappings (13 x 8):** each locale's placeholders, metadata, redirects and `data/*` sheets are mapped explicitly to their locale path without an extension; the system appends `.json` (e.g. `/de-de/placeholders.json`, `/de-de/metadata.json`, `/de-de/data/authors.json`).
- **13 taxonomy mappings:** a taxonomy page isn't a spreadsheet, so its mapping includes the extension to publish it as JSON, e.g. `/content/tcs-corp/de/de-de/taxonomy:/de-de/taxonomy.json`.
- **No site-root redirects sheet:** each locale's redirects sheet is published at `/{code}/redirects.json`, but the platform only applies a root `/redirects.json`, so they aren't applied automatically (see *Known Gaps*).
- **No root `/metadata.json`:** bulk metadata is symmetric across the 13 locales; each locale's sheet stays at its own path (`/en/metadata.json`, `/de-de/metadata.json`, ...). The platform applies them only once **all 13** are listed in the site configuration's `metadata.source` (`POST https://admin.hlx.page/config/{org}/sites/{site}/metadata.json` with `{"source": ["/en/metadata.json", "/de-de/metadata.json", ...]}`); until then **no bulk metadata applies**. Each sheet's URL patterns must start with its locale (e.g. `/de-de/**`).
- `includes`: the 13 locale roots.
- `excludes`: the 13 `{country}/{lang}/block-library` paths (and `/**`) and `site-global-templates` (and `/**`); they're used on author only.
- **No root catch-all:** nothing is published at `/`, so the site root needs a `/` -> `/en/` redirect through another mechanism, e.g. a CDN rule.
- **After changing `LOCALES`**, run `npm run build:paths`, commit `utils/site.js` and `paths.json` together, and update `metadata.source` (see *Add a Locale*).

Query Index (helix-query.yaml)

`/query-index.json` excludes non-content pages of every locale (`/*/references/**`, `/*/error-pages/**`, `/*/data/**`, `/*/nav`, `/*/footer`, `/**/fragments/**`, `/**/modals/**`; `/*/block-library(/**)` and `/site-global-templates/**` as a safeguard) and indexes `lastModified`, `robots`, `title`, `description`, `image`, `tags` and the metadata sheet columns `metaTitle`, `metaDescription`, `pageType`, `contentType`, `authorId`, `publishDate`, `modifiedDate`, `readingTime`, `thumbnailImage`, `featuredImage`, `region`, `country`, `language`, `persona`, `campaignId`, `featured`, `priority`. The Conversation Canvas menu builds its site tree from this index, rooted at the locale (e.g. `/de-de`) by default.

Conventions

- **Page properties are limited** to Title, Description, Keywords, Page Type, Navigation and Footer. Other page-level data is a metadata sheet column. Page-specific values that aren't metadata (CTAs, author image overrides, share / download) are **block fields**.

- **Locale-aware paths.** Never fetch root paths such as `/nav` or `/placeholders.json`, and never build `/{country}/{lang}` paths by hand; use `localePagePath()`, `resolvePagePath()`, `localeSheetPath()`, `toPublishedPath()` / `toAemPath()` from `utils/site.js`, so code works both on the published site (`/de-de/...`) and in Universal Editor (`/content/tcs-corp/de/de-de/...`).
- **No hard-coded UI strings.** Every user-facing string, including ARIA labels and status messages, goes through `getPlaceholders()` with an English fallback.
- **Decorators for UI, utils for everything else.** Every button and link button goes through the button decorators. Decorators may import utils. **Utils never import decorators.** Keep one purpose per file and add new shared behavior there, not in a block. Each folder's README lists its modules, exports and users.
- **Keep editor instrumentation.** When rebuilding markup, use `moveInstrumentation()` or `fromCell()`.
- **Read site data through the utils**, not `getMetadata()` or `fetch()` directly in blocks, so caching and fallbacks stay in one place.

Shared Code: Decorators and Utils

Decorator	Exports	Used by
`button-decorator.js`	`decorateButton(el, { view, type, icon, title, newWindow, disabled })`, `createButton`, `setButtonDisabled`, `setButtonAlignment`, `removeButtonDecoration`	most blocks, header
`icon-button-decorator.js`	`createIconButton`	gen-carousel, page-header, modal, embed, video, conversation-canvas
`content-buttons-decorator.js`	`decorateButtons(container)`: standalone authored links become buttons	`scripts.js`, `editor-support.js`
`link-decorator.js`	`setLinkTarget` (open in a new window)	button decorator, teaser
`heading-decorator.js`	`buildHeading` (Title + Title Type), `HEADINGS`	teaser, quote, testimonial, form-variations
`content-decorator.js`	`fromCell` (keeps editor instrumentation)	teaser, quote, testimonial, page-header, form-variations
`image-decorator.js`	`pictureOf` (image + alt text field)	teaser, quote, testimonial, page-header
`responsive-image-decorator.js`	`buildImage` (image from a URL; site `/media_` optimized, other URLs lazy `<img>`)	`scripts.js` (section backgrounds), conversation-canvas
`date-decorator.js`	`buildTime`	teaser, page-header
`tag-decorator.js`	`buildTagList`	teaser, page-header
`media-decorator.js`	`createFrame`, `createYoutubeFrame`, `createVimeoFrame`, `createVideoElement`	embed, video
`seo-decorator.js`	`decorateHead` (SEO / social metadata columns to standard head tags, `<html lang>`)	`scripts.js`

Decorator	Exports	Used by
social-links-decorator.js	buildSocialLinks (Social Links sheet as an icon list; icon names from /icons or image URLs)	footer
template-decorator.js	ensurePageHeader, appendLoadedSection	article, blog, press-release templates
author-bio-decorator.js	buildAuthorBio	article / blog template
contact-list-decorator.js	buildContactList (mailto / tel links)	press-release template
a11y-decorator.js	hideVisually, createHiddenText	teaser, testimonial, table-list, conversation-canvas, social links

Util	Exports	Used by
site.js	LOCALES, DEFAULT_LOCALE, getLocale, toPublishedPath, toAemPath, localePagePath, resolvePagePath, localeSheetPath, sheetRows	header, footer, scripts.js, placeholders, taxonomy, data, conversation-canvas
metadata.js	getPageMetadata, getPageTags, getPageLanguage	page-header, conversation-canvas, SEO decorator, templates
placeholders.js	getPlaceholders, loadPlaceholders, fillPlaceholder	all blocks with UI text, header, footer, templates
taxonomy.js	loadTaxonomy, resolveTags	teaser, page-header, conversation-canvas
data.js	loadDataSheet, getAuthor, getAuthors, getLocations, getPartners, getSocialLinks, getMediaContacts, inRegion	page-header, footer, templates
properties.js	readProperties, textOf, isOn, optionOf, normalizeKey, textSource	key-value blocks, decorators
multifield.js	readMultifieldItems (items separated by <hr> or list items)	links util, testimonial, anchor-list
links.js	NEW_WINDOW, linkOf, readLinkItems	hero, teaser, page-header
buttons.js	BUTTON_VIEWS, BUTTON_TYPES, BUTTON_ALIGNMENTS, pick, readButtonOptions	button decorators, hero, list, conversation-canvas
dates.js	formatDate (DD-MM-YYYY, MM-DD-YYYY, MMM d, yyyy, DD/MM/YY; no time zone shift)	date decorator
tags.js	readTagValues (tag IDs as authored), parseTag, readTags (ID-derived labels)	teaser, taxonomy util
urls.js	toSitePath (AEM /content/... paths to site paths)	table-list, conversation-canvas
ids.js	nextInstance, uniqueId, createIdFactory	tabs, gen-carousel, page-header, form-variations, conversation-canvas
environment.js	prefersReducedMotion, scrollBehavior, isEditing, isInEditor	gen-carousel, hero, table-list, anchor-list, video, form-variations
viewport.js	onVisible (once), watchVisibility	gen-carousel, hero, table-list, embed, video

Util	Exports	Used by
media.js	isYoutube, isVimeo, isAnimation, youtubeEmbedUrl, vimeoEmbedUrl, videoType	media decorator, embed, video, hero
lottie.js	loadLottie(): loads /scripts/lib/lottie/lottie_light.min.js once per page	hero

Button Vocabulary

Authored buttons (from the **Button** component, `models/_button.json`) and content links follow one vocabulary:

- **Views:** `button`, `icon`, `link`
- **Types:** `primary`, `secondary`, `tertiary`
- **Icons:** `arrow-right`, `chevron-right`, `chevron-down`, `search`
- **Alignment:** `left`, `center`, `right` (classes `button-align-*`)

`readButtonOptions()` reads the options from the markup. **Bold** means primary, *italic* means secondary, and bold + italic means tertiary. A link that contains only an icon gets the icon view. A type class or `view-*` / `align-*` classes or data values on the link or its paragraph also work. A standalone link without formatting becomes a **link button** (text + chevron). Inline links and links that show their own URL are left alone.

Local Development Setup

Prerequisites

- Node.js (no `.nvmrc` or `engines` field; use a current LTS release)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
git clone https://github.com/sradha-tcs/tcs-corp.git
cd tcs-corp
npm install
aem up            # or: npx -y @adobe/aem-cli up --no-open --forward-browser-logs
```

Local Server

- URL: http://localhost:3000, with auto-reload on file changes
- Serves your local code with **previewed** content, including the locale sheets (`/en/placeholders.json`, `/de-de/taxonomy.json`, `/en/data/*.json`, ...) and `/query-index.json` once they are published
- Test content without CMS pages: put static HTML in `drafts/` and run `aem up --html-folder drafts`. Add `<meta>` tags (`page-type`, `article:tag`, `publish-date`, `author-id`, `nav`, `footer`) to the draft's head to test metadata-driven code. Draft pages under `/drafts/` use the default locale `en`. Test another locale with a static page under its root (e.g. `drafts/de-de/test.html`, served at `/de-de/test`)
- Inspect markup: `curl http://localhost:3000/{path}.plain.html` or `.md`

Linting and Model Build

```
npm run lint        # ESLint (JS + JSON models) and Stylelint
npm run lint:fix
```

```
npm run build:json  # regenerate component-definition/-models/-filters.json
npm run build:paths # regenerate paths.json from LOCALES in utils/site.js
```

Run `npm run build:json` after **every** change to `models/_*.json` or `blocks/*/_*.json`, and commit the generated files. `npm run lint` passed when this guide was generated.

`xwalk/max-cells` overrides (`.eslintrc.js`): The default limit is 4 cells. Overrides: `section: 6` (section options render as section-metadata rows, not block cells), `table-list: 10`, `table-list-card: 6`, `table-col-5: 5`, `field-text`, `field-select` and `field-textarea: 6`, `field-checkbox: 5`, `contact-card: 5`. Raise a limit only when a model really needs one cell per value. Other enforced rules: `import/extensions` (always `.js`), `linebreak-style: unix`, `no-param-reassign` with `props: false`.

Design System

All tokens live in `:root` in `styles/styles.css` (about 220 custom properties). They come from the TCS.com Design System (Figma).

Primitive Colors

```
--brand-200: #cedff4;  --brand-300: #a7c8f1;  --brand-400: #216ac3;
--brand-500: #4673db;  --brand-600: #011627;
--grey-100 ... --grey-1000   /* #f5f5f5 ... #111 */
--slate-100 ... --slate-1000 /* #f3f3f3 ... #242424 */
--black-100 ... --black-1000 /* #0c0c0d at 5% ... 100% alpha */
--white-100 ... --white-1000 /* #fff at 5% ... 100% alpha */
```

Semantic Colors (use these in blocks)

Group	Tokens
Background	`--color-bg-default`, `-secondary`, `-tertiary`, `-opacity`, `-disabled`, `-brand`, `-brand-secondary`, `-brand-tertiary`, `-neutral`, `-neutral-secondary`, `-neutral-tertiary` (most have `-hover`)
Text	`--color-text-default`, `-secondary`, `-tertiary`, `-disabled`, `-brand`, `-brand-secondary`, `-on-brand`, `-neutral`, `-neutral-secondary`, `-neutral-tertiary`, `-on-neutral`, `-overlay`, `-light-overlay`
Icon	`--color-icon-default`, `-secondary`, `-tertiary`, `-disabled`, `-brand`, `-brand-secondary`, `-on-brand`, `-on-disabled`, `-neutral`, `-on-neutral`
Border	`--color-border-default`, `-secondary`, `-tertiary`, `-disabled`, `-brand`, `-brand-secondary`, `-brand-tertiary`, `-neutral`, `-neutral-secondary`, `-neutral-tertiary`
Boilerplate aliases	`--background-color`, `--light-color`, `--dark-color`, `--text-color`, `--link-color`, `--link-hover-color`

Typography

```
--body-font-family: inter, inter-fallback, sans-serif;
--heading-font-family: var(--body-font-family);
--code-font-family: 'Roboto Mono', ui-monospace, menlo, consolas, monospace;
```

```
--font-weight-light: 300; --font-weight-regular: 400; --font-weight-medium: 500;
--font-weight-semibold: 600; --font-weight-bold: 700;
```

Token	Mobile	Desktop (>= 900px)
`--display-font-size-1 / 2 / 3`	44 / 40 / 36px	80 / 64 / 40px
`--heading-font-size-xxl / xl / l`	32 / 28 / 24px	48 / 40 / 28px
`--heading-font-size-m / s / xs`	18 / 16 / 14px	24 / 20 / 16px
`--body-font-size-l / m / s / xs`	16 / 16 / 12 / 12px	24 / 18 / 16 / 14px
`--label-font-size`, `--button-font-size`	14px, 16px	same

Line heights: `--display-line-height: 1.2`, `--body-line-height: 1.6`, `--label-line-height: 1.4`.

Spacing, Layout and Shape

```
--space-0 ... --space-6000   /* 0, 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96, 120, 160,
200, 240px */
--spacing-xs: 12px; --spacing-s: 24px; --spacing-m: 40px; --spacing-l: 80px; --spacing-xl:
200px;
--page-max-width: 1920px; --content-max-width: 1440px; --nav-height: 64px;
--section-padding-inline: 24px (600px: 40px, 1200px: 80px);
--section-spacing: 40px (900px: 80px, 1200px: 120px);
--radius-s: 4px; --radius-m: 8px; --radius-l: 16px; --radius-xl: 24px; --radius-full:
9999px;
--border-width-100/200/300/500: 1/2/3/5px;
--icon-size-s/m/l: 24/32/40px; --button-height: 50px; --icon-button-size: 44px;
--shadow-100 ... --shadow-600, --shadow-inner-100 ... -600, --blur-100: 4px;
```

Breakpoints

Name	Min-Width	Usage
Mobile	0	Default styles
Tablet	600px	`@media (width >= 600px)`
Desktop	900px	`@media (width >= 900px)`
Large	1200px	`@media (width >= 1200px)`

The project uses media query **range syntax** (`width >= 600px`) and standard breakpoints. Write styles mobile-first.

Fonts

Family	Weights	File	Usage
Inter (variable)	300-700	`fonts/inter-variable.woff2`	Body and headings

Family	Weights	File	Usage
		(Latin subset, `font-display: swap`)	
inter-fallback	n/a	`local('Arial')` with size-adjust and metric overrides	Prevents CLS while Inter loads

`Roboto Mono` in `--code-font-family` is **not** shipped, so code text falls back to system monospace fonts.

Global Components in styles.css

- **Buttons:** `a.button` / `button.button` with `.primary` (black pill), `.secondary` (outline), `.tertiary` (text + arrow). The trailing arrow is a CSS mask (`icons/arrow-right.svg`). Disabled state uses `aria-disabled="true"` or `:disabled`.
- **Icon buttons:** `.button.icon-button` takes its icon from the `--icon-url` mask. It has primary, secondary and tertiary types.
- **Link button:** `main a.link-button` (underlined, chevron).
- **Tags:** `.tag`, `.tag-filter` (pill, `.active` / `[aria-current]`), `.tag-card` (small caps), `.tag-industry` (`| text`). Themed through `--tag-*` variables.
- **Avatars:** `.avatar` (large / `.medium` / `.small`, `.square`), `.avatar-group` (`.overlap`), `.avatar-overflow` (+N).
- **Accessibility:** `.visually-hidden`. Focus ring: 2px `--color-border-brand`.

Section Styles

Class	Effect
`highlight` / `light`	`--color-bg-secondary` background, padded
`dark`	Neutral (black) background, light text. Buttons and link buttons are re-themed
`brand`	Light brand background (`--color-bg-brand-secondary-hover`)
`fixed-width`, `rounded`, `has-background`	Set by the section options (see above)
`gen-carousel-section`	Gen Carousel section

> **Note:** The Section **Style** field offers only **Highlight**. The `light`, `dark` and `brand` styles exist in CSS but authors can't select them until they are added to `models/_section.json`.

Blocks Reference

Twenty-one blocks were built or adapted for TCS (including `header` and `footer`, described at the end of this section). Most are based on Figma frames from the TCS.com Design System. The boilerplate blocks `cards`, `columns` and `fragment` are unchanged (`fragment` provides `loadFragment()` for Modal). All UI text in blocks comes from placeholders (see *Site Data*).

Content and Hero Blocks

Block	Purpose	Variants / Options	Key Features
hero	Page hero with icon, eyebrow, title (h1/h2), description, image or Lottie animation, and CTA list	`lottie` (Hero Image type "Lotto"), `align-top`, `align-middle`, `align-bottom`, `bg-dark`, `bg-dark-subtle`, `bg-black`	Rows can come in any order and are sorted into content, media and CTA cells. The Lottie player loads only near the viewport and when the browser is idle, pauses off screen, and shows the first frame for reduced motion. CTA view is set by bold or italic formatting. `new-window` link type
teaser	Key-value "2 Slot Section" / "Text & Image" / "Text Section"	Style `no-image`. BG `grey`, `dark`, `dark-green`. Image left or right. `personalization` flag	Show/hide toggles for eyebrow, title, description, image, date (3 formats) and tags (AEM tags, **labels from the taxonomy**; decoration is async). Multi-links or primary/secondary CTA, with link styles Default, `list`, `primary` and `secondary`. View-all link
page-header	Page header. **Title, description, tags, publication date, author and template come from the page metadata**; everything else is a block field	Layouts `landing` and `article`. Without a block Style, Template `landing` / `listing` gives `landing`, anything else `article`	See *Page Header Fields* below
quote	Quote with eyebrow, title, author and optional image	BG `grey`. `quote-has-media`	Title Type h1-h6 via `buildHeading`
testimonial	Testimonial with author image, designation and location, and a stats multifield	BG `grey`. Hide eyebrow or title, show description	Stats are composite multifield items (title + value)
list	Header row with title (h1-h6) and an optional CTA	CTA style `primary` or `secondary`	`list-header-row-no-cta` when there is no link

Page Header Fields

The model (`blocks/page-header/_page-header.json`) has four tabs:

Tab	Fields	Rendering
Style	`classes` (landing / article)	Overrides the Page Type-based layout (`landing` / `listing`: landing, otherwise article)
CTAs	`primaryCta` + `primaryCtaText`, `primaryCtaTarget`, `primaryCtaStyle`; the same for `secondaryCta`	Primary defaults to the `primary` button. Secondary defaults to a link button. Style `primary` / `secondary` makes a button. Target `new-window` opens a new window
Author & Meta	`authorName`, `authorImage` + `authorImageAlt`, `designationLocation`, `readingTime`, `collectionLabel`, `listenLink`, `socialShare`, `download`	Article layout only. Without block values, the author name, image, designation and region come from the Authors sheet entry of the page's `author-id` (the name links to LinkedIn), then the `author` metadata; the reading time comes from the `reading-time` metadata. A numeric reading time renders as `minRead` ("{n} min read"). Listen is a tertiary icon link (headphones). Download calls `window.print()`. Share uses `navigator.share`,

Tab	Fields	Rendering
		falling back to the clipboard with a `role="status"` message
Image Collage	`collageImage1..3` + alt texts	Landing layout only. All text on the left, collage on the right

The article meta row shows tags (taxonomy labels), the collection label, "Published {date}" (`dd/mm/yy` from `publish-date`, falling back to `publication-date`), reading time and the actions. The title is always an `h1` with the authored page title (not the SEO `meta-title`). Article, blog and press-release templates add a Page Header automatically when none is authored.

Navigation and Collection Blocks

Block	Purpose	Variants / Options	Key Features
gen-carousel	Carousel built automatically from a Gen Carousel section (see *Gen Carousel Sections*)	`manual`, `auto`, `manual-auto`. Interval 2-30 s (default 5)	ARIA carousel region. Prev/next, dots, swipe and pause, all labelled through placeholders. Doesn't depend on slide markup. Turns off autoplay while editing
anchor-list	Chapter navigation to sections with an **Anchor name**	`sticky`	Leaves room for a sticky list or a fixed mobile header when scrolling. Honors reduced motion. Mobile label `onThisPage`
table-list	Heading with "View all" and up to N numbered cards, shown as a list or a table	Variations `compact`, `no-description`, `dark`, `featured-industry`, `no-number`, `description-first`, `title-right`. Card color white, blue, black or grey. Display title-description, title or description. Motion none, fade, slide or reveal	Properties are read **by position** (one row per field, max-cells 10). Resolves site paths, AEM paths, anchors and URLs. Motion is skipped in the editor and for reduced motion
tabs	Tabs with heading (h3-h6), image and rich text per tab	n/a	Unique ids per instance (`nextInstance`)
accordion	Summary/text items	n/a	Block Collection based, keeps editor instrumentation
table	Table with 1-5 columns	Columns filter `table ... table-5-columns`	Block Collection based, separate row models per column count

Media, Forms and Interactive Blocks

Block	Purpose	Variants / Options	Key Features
conversation-canvas	AI "conversation" search canvas with site-tree menu and search bar	Default, `primary` (sticky floating dock), `secondary`. Title size `bigger` or `medium`. Background image	Menu loaded from `/query-index.json` (root path + depth). **Suggestion pills default to the page's Tags** (taxonomy labels) and can be replaced through `setSuggestions()`. Speech input through `SpeechRecognition` when available. Submitting dispatches a **`conversation-canvas:search`** event with `{ query }`. **There is no search backend yet**
form-variations	Configurable form: text/email/tel, select,	Theme `dark`	Conditional fields (`dependsOn` a dropdown value). Hidden fields are

Block	Purpose	Variants / Options	Key Features
	textarea (max length), consent checkbox, contact cards and submit		disabled and not required. Everything stays visible in the editor. **There is no submit endpoint**: a valid submit dispatches **`form-variations:submit`** with `{ data }` and shows `formSuccess`
embed	YouTube, Vimeo or generic iframe embed with placeholder image	n/a	Loads on visibility or click. Play button from `createIconButton`
video	Video (YouTube, Vimeo or file) with placeholder	Options multiselect	Respects reduced motion. Lazy-loaded with `onVisible`
modal	Opens a fragment in a `<dialog>`. The authored block renders a button	n/a	`createModal()` / `openModal()` exported. `/modals/` links open automatically
filter-list	**Placeholder:** renders nothing. Universal Editor shows a labelled placeholder	n/a	Dialog and logic still to be built
interactive-dashboard	**Placeholder:** renders nothing. Universal Editor shows a labelled placeholder	n/a	Dialog and logic still to be built

Header: boilerplate, plus these changes. The nav page comes from `resolvePagePath(getMetadata('nav'), 'references/nav')`, the logo link is turned back into a plain link with `removeButtonDecoration()`, and the hamburger and navigation labels come from placeholders.

Footer: the footer page comes from `resolvePagePath(getMetadata('footer'), 'references/footer')`; the locale's Social Links sheet is appended as `.footer-social` (icons for linkedin, x / twitter, facebook, instagram, youtube; other platforms as text).

Block Implementation Patterns

- **Key-value blocks** (teaser, quote, testimonial, page-header, conversation-canvas, form-variations) read rows with `readProperties()` / `textOf()` / `optionOf()`. Keep field names in the block's `_*.json` in sync with the keys in the JS (keys are normalized: lowercase, no separators).
- **Composite multifields** (hero CTA list, testimonial stats, anchor-list links) are read with `readMultifieldItems()`.
- **Async decoration:** blocks that load placeholders or the taxonomy export `async function decorate()` and load both in parallel (`Promise.all`).
- When rebuilding markup, always keep Universal Editor instrumentation (`moveInstrumentation`, `fromCell`).
- Use `isEditing()` / `isInEditor()` to turn off motion and hiding inside Universal Editor.

Integrations and Delayed Phase

`scripts/delayed.js` is unchanged from the boilerplate. **No analytics, tag manager, consent or chat integrations are set up yet.** Add them to `delayed.js`, never to `scripts.js`. The only third-party code is the vendored Lottie player and DOMPurify, both loaded on demand. There are no CDN imports.

The blocks expose two integration points for future backends:

```
document.addEventListener('conversation-canvas:search', (e) => { /* e.detail.query */ });
document.addEventListener('form-variations:submit', (e) => { /* e.detail.data */ });
```

Universal Editor Models

File	Customization
`models/_page.json`	Title, Description, Keywords, **Page Type** (Default, Home, Article, Blog, Press Release; published as `page-type`), **Navigation** and **Footer** (content pickers; published as `nav` / `footer`)
`models/_section.json`	Section Name, Style (Highlight), Width Type, Rounded edge, Background Image + alt, Anchor name. The filter lists all the TCS blocks
`models/_button.json`	Link, Text, Title, Views, Type (conditional), Icon, Alignment
`models/_component-filters.json`	Main filter allows `section` and `gen-carousel`
`blocks/page-header/_page-header.json`	Style, CTAs, Author & Meta and Image Collage tabs (see *Page Header Fields*)
`paths.json`	Maps the 13 locale folders to their language roots (`us/en` -> `/en`, `de/de-de` -> `/de-de`, ...) their spreadsheets and their taxonomy pages; generated by `npm run build:paths`; excludes the 13 block libraries and the template pages

Other page-level data (author-id, publish-date, reading-time, region, SEO and social values, ...) is set **in the locale's bulk metadata sheet**, not as page-model fields.

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/{name}.js`, `{name}.css` and `_{name}.json` (definition, model, filter).
2. Add the block id to the `section` filter in `models/_section.json`, or to `gen-carousel` if it can be a slide.
3. Implement `export default async function decorate(block)`. Use **decorators** for buttons, headings, images, tags and media, and **utils** for properties and site data.
4. Get every user-facing string from `getPlaceholders()` with an English fallback, and add the new keys to the placeholders sheet.
5. Scope all CSS to `.{name}` and use semantic tokens (`--color-*`, `--space-*`).
6. Run `npm run build:json` and `npm run lint`, then test at http://localhost:3000 and in Universal Editor.

Add a Template

1. Create the template page under Site Global Templates in AEM; its name is the Page Type (e.g. `case-study`).
2. Add `templates/case-study/case-study.js` (default export `decorate(main)`, runs before decoration) and `case-study.css` (scope selectors to the added content or `body.case-study`).
3. Add the name to `TEMPLATES` in `scripts/scripts.js` and as a Page Type option in `models/_page.json`, then run `npm run build:json`.
4. Build added content with decorators (`ensurePageHeader`, `appendLoadedSection`) and data from utils; never block LCP on sheet data.

Add a Locale

1. Add `{ country, code, lang }` to `LOCALES` in `utils/site.js` (e.g. `{ country: 'it', code: 'it-it', lang: 'it-IT' }`).
2. Run `npm run build:paths` to regenerate `paths.json` (locale folder, sheet and taxonomy mappings, include and block-library excludes), and commit `utils/site.js` and `paths.json` together.
3. Apply `paths.json` through the configuration service and add the locale's metadata sheet (`/it-it/metadata.json`) to `metadata.source` (Admin API, `POST https://admin.hlx.page/config/{org}/sites/{site}/metadata.json` with the full list of locale sheets).
4. In AEM, create `/content/tcs-corp/it/it-it/` with `references/nav`, `references/footer`, `error-pages/404`, `placeholders`, `taxonomy` (a sheet named by the ISO 639-1 code, e.g. `it`, for translated titles), `metadata` (URL patterns starting with `/it-it/`), `redirects`, `data/*` and `block-library/`.
5. `helix-query.yaml` needs no change (its `/*/...` excludes cover every locale). Run `npm run lint` and check `/it-it/` on preview.

Use a Data Sheet

Read rows through `utils/data.js` (add a typed reader there for a new sheet under `data/`); render them with a decorator. Sheets are per locale, so use `localeSheetPath()` for any new sheet.

Add UI Text

1. In code: `const t = await getPlaceholders();` then `t('myKey', 'English text', { var })`.
2. In content: add a row (Key `myKey` or `My Key`, Text) to the placeholders sheet and publish it.

Use Page-Level Data

1. If it is page metadata (a metadata sheet column such as page-type, author-id, publish-date, region, ...), read it with `getPageMetadata()` / `getPageTags()`. For a new column, add it to `getPageMetadata()`, to `helix-query.yaml` if listings need it, and to the metadata sheet.
2. If it is specific to one component (a CTA, an image, a toggle), add it as a **block field**. Don't add page properties beyond the agreed set.

Add a Section Option

1. Add the field to `models/_section.json`.
2. Add its normalized key (lowercase, no dashes) to `SECTION_OPTIONS` in `scripts.js` and apply it in `decorateSectionsWithOptions`.
3. Check the `section` max-cells limit in `.eslintrc.js`.

Modify Global Styles

Edit `styles/styles.css` and use tokens. Test pages with dark and highlight sections, and watch for CLS.

Add Analytics or a Marketing Tool

Add it to `scripts/delayed.js` and check in the Network tab that it loads about 3 s after page load. Scripts must work with the CSP in `head.html` (`strict-dynamic`, nonce `aem`). Load them with `loadScript()` from already-trusted scripts.

Debug a Block

1. Check the browser console. `aem up --forward-browser-logs` forwards it to the terminal.
2. Compare the delivered markup (`curl .../{path}.plain.html`) with what the block expects (row order, keys).

3. Check the page `<head>` for the metadata the block reads (`page-type`, `article:tag`, `publish-date`, `author-id`, `reading-time`, `nav`, `footer`).
4. Check that variant classes and the `data-block-status` attribute are applied.
5. In Universal Editor, check that `data-aue-*` attributes survived decoration.

Environments

Environment	URL Pattern	Purpose
Local	http://localhost:3000	Development
Feature Branch	`https://{branch}--tcs-corp--sradha-tcs.aem.page/`	PR testing
Preview	https://main–tcs-corp–sradha-tcs.aem.page/	Staging
Live	https://main–tcs-corp–sradha-tcs.aem.live/	Production

Git Workflow

Branch Naming

- Features: `feature/tcs-{description}` (the convention so far, e.g. `feature/tcs-decorators-utils`)
- Fixes: `fix/{description}`
- Keep names short, because they become part of the preview hostname.

PR Requirements

1. Include a feature preview URL (`https://{branch}--tcs-corp--sradha-tcs.aem.page/{path}`) that shows the change. PRs without one are rejected.
2. Pass `npm run lint` (the Husky pre-commit hook runs it) and `gh pr checks`.
3. Run PageSpeed Insights on the feature preview URL, aiming for 100.
4. No console errors, and test in Universal Editor as well as on the published page.

Troubleshooting

Block not loading?

1. The block folder name must match the class name.
2. The JS must `export default function decorate` (async is fine).
3. Look for 404s in the Network tab.
4. In Gen Carousel sections, child blocks are decorated by `buildCarouselSections()`, so check that the section has the `gen-carousel-section` style.

Component missing in Universal Editor?

1. Run `npm run build:json` and commit the `component-*.json` files.
2. Make sure the component id is in the right filter (`section`, `gen-carousel` or the block's own filter).
3. `npm run lint` reports `xwalk/max-cells` and other model rule violations.

UI text doesn't change after editing the placeholders sheet?

1. Check that the locale's placeholders sheet (e.g. `/de-de/placeholders.json`) is published and returns `Key` / `Text` columns.
2. The key must match after normalization (case and separators are ignored, other characters aren't).
3. Placeholders are cached per page load, so reload the page.

Tags show labels like "Cloud" instead of the taxonomy title?

That's the ID-derived fallback: the tag ID is missing from the locale's taxonomy (e.g. `/de-de/taxonomy.json`; translated titles in the sheet named by the ISO 639-1 language, e.g. `de`), or the taxonomy isn't published. Publish the taxonomy and check the exact tag ID.

Page Header shows the wrong layout or no date or author?

Check the page metadata: `page-type` (`landing` / `listing` for the landing layout), `publish-date` and `author-id` (which must match an `author-id` in the locale's `data/authors` sheet). Set them in the metadata sheet or on the page, or set the block Style to force a layout.

Header or footer missing?

Check that the locale's `references/nav` / `references/footer` are published, or that the configured Navigation / Footer page property or the `nav` / `footer` sheet column points to a published page (`curl {path}.plain.html`).

Template not applied?

The Page Type must be in `TEMPLATES` (`scripts.js`) and have a `templates/{name}/` folder; check the `page-type` meta in the page head and the network tab for the template JS / CSS.

Section option ignored?

Section options must be in `SECTION_OPTIONS`, as normalized keys.

Styles not applying?

Check specificity: buttons use `a.button.primary`, and dark sections override buttons with `main .section.dark ...`. Also check that the file loads and that Stylelint passes.

Content not updating?

Preview the page again in Universal Editor (publish to preview), clear the browser cache, and wait 1-2 minutes for the CDN.

Known Gaps and Gotchas

- **Site data not published yet:** when this guide was generated, the `/en/` pages and sheets (references, error pages, placeholders, taxonomy, data) returned **404** on preview. Header, footer and error pages need `references/nav`, `references/footer` and `error-pages/404` to be published; the site falls back to English text and ID-derived tag labels.
- **Config Service:** The org name `sradha-tcs` is a placeholder, and the sites API needs authentication (401). Apply `paths.json` through the configuration service and reindex the query index after the changes.
- **Social previews:** SEO / social columns with custom names are applied in the browser only. Add standard columns (`og:title`, `og:description`, `og:image`, `twitter:card`, `canonical`) for crawlers that don't run JavaScript.

- **Bulk metadata not applied yet:** there is no root `/metadata.json`; no bulk metadata applies until all 13 locale sheets (`/en/metadata.json`, `/de-de/metadata.json`, ...) are listed in the site configuration's `metadata.source` (Admin API). When a locale is added, its sheet must be added to `metadata.source` too.
- **Redirects not applied:** each locale's redirects sheet is published at `/{code}/redirects.json`, but the platform only applies a root `/redirects.json`, and there is none. Redirects need another mechanism, e.g. CDN rules.
- **Site root:** `paths.json` has no root catch-all, so `/` returns 404 until a `/` -> `/en/` redirect is set up outside the redirects sheets (e.g. a CDN rule).
- **Old `/us/en/` URLs** are no longer published (English is now `/en`); redirect them the same way (e.g. CDN rules) if any were shared.
- **Error pages 403 / 500:** only the 404 is served automatically (`404.html`); 403 and 500 must be configured in the CDN.
- **Unused data sheets:** Locations and Partners have readers (`utils/data.js`) but no block displays them yet.
- **Placeholder blocks:** `filter-list` and `interactive-dashboard` render nothing yet.
- **No backends:** Form Variations and Conversation Canvas only dispatch DOM events.
- **Section styles:** `dark`, `light` and `brand` are styled but not offered in the section Style field.
- **No analytics** are set up in `delayed.js`.
- **Removed APIs:** `scripts/page-properties.js` (`getPageProperties`, `applyPageProperties`) no longer exists. Use `utils/metadata.js`; the Page Type is now a plain body class (e.g. `press-release`).
- `buildAutoBlocks()` is still an empty TODO. Gen Carousel auto-blocking happens in `decorateSectionsWithOptions()`.
- `package.json` still uses the boilerplate name `@adobe/aem-boilerplate`, and there is no Node version pin.
- `scripts/aem.js` must never be modified (per `AGENTS.md`).

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Component Model Definitions (xwalk)	https://www.aem.live/developer/component-model-definitions
Special Metadata Properties	https://www.aem.live/docs/special-metadata-properties
Bulk Metadata	https://www.aem.live/docs/bulk-metadata
Placeholders	https://www.aem.live/docs/placeholders
Taxonomy	https://www.aem.live/docs/authoring-taxonomy
Redirects	https://www.aem.live/docs/redirects
Path Mapping (AEM authoring)	https://www.aem.live/developer/authoring-path-mapping
Error Pages	https://www.aem.live/docs/error-pages
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading / Keeping it 100	https://www.aem.live/developer/keeping-it-100

Resource	URL
Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices
xwalk ESLint rules	https://github.com/adobe-rnd/eslint-plugin-xwalk
Boilerplate (xwalk)	https://github.com/adobe-rnd/aem-boilerplate-xwalk

Support Contacts

Role	Contact
Repository owner	sradha-tcs (GitHub)
Technical lead	*To be added*
AEM / Universal Editor administrator	*To be added*